

Bio-inspired Artificial Intelligence – Project 1

Johannes Lorentzen
Svein Kåre Sørstad

09.02.2026

Introduction

This report documents the work performed by the authors for project 1 in the subject IT3708 Bio-inspired Artificial Intelligence. The theory section will cover the basic concepts of the genetic algorithms used in the project, and the results section will cover the actual experiments that were performed and the resulting data. This is done due to the page restriction for the project, which only allows a maximum of three pages. In this project, the group implemented both a simple genetic algorithm, as well as a crowding algorithm, to solve a feature selection problem. The problem consisted of a dataset with 101 features and one target, where the features were used for a linear regression model. The goal was to find a group of features that would result in an equal or lower RMSE, which is a minimization problem where a higher/better fitness means a lower value of RMSE.

Theory

A simple genetic algorithm is an algorithm that simulates a population of individuals, where the principle of “survival of the fittest” is applied to problems in order to estimate a good solution. Each individual in the population is a possible solution, where some objective function is used to compute a fitness value. The fittest individuals are then “mated” through recombination and mutation to produce offspring for the next generation. By running this process for enough generations, this process eventually produces better solutions than what the population started with (Mengshoel, 2026).

In order to escape local optima in a population of solutions, different crowding methods can be used to increase the diversity of the population. Crowding is similar to the simple genetic algorithm, but instead of simply replacing the population with the offspring, there is a tournament step. This tournament step involves pairing the two parents with the child that is most similar to them according to some measure, typically Hamming distance for bitstrings, and then performing a fitness-based selection on the pairs. In this way, the parents can still survive to the next generation if they are more fit than their offspring. This allows for a greater diversity of solutions, allowing the algorithm to explore multiple local optima (Díaz, 2026).

Results

This section will cover the implementations required for the subtasks f_i in the project description. Each paragraph covers the respective subtask in the same order as the project description.

Running the linear regression with all features using the python code from the handout produces a RMSE of 0.1357 using the seed 1234. Figure 1 shows the results of using a simple genetic algorithm, which resulted in a RMSE that was slightly higher, although using fewer features. The figure shows that the fitness (RMSE) starts out at 0.15-0.155 and steadily declines until it converges around 0.143. The algorithm was run for 50 generations, with a population size of 100, mutation rate of 0.01 and crossover rate of 0.85.

Building on this result, the group tried the algorithm with differing parameters. Although the RMSE varied significantly across attempts, it seemed that a higher value of k for the tournament parent selection led to a faster improvement in fitness. Throughout the runs, the lowest value observed was around 0.119, and the highest values were around 0.15-0.16. This was achieved using both the SGA and the crowding algorithms. A higher k in tournament selection increases selection pressure, so it seems natural that this would lead to better fit solutions. On the other hand, altering crossover rate and mutation rate did not seem to produce any consistently different

results. Altogether, it seemed that the initial population heavily influenced the results, with the final solution only being a slight improvement over the initial best fitness.

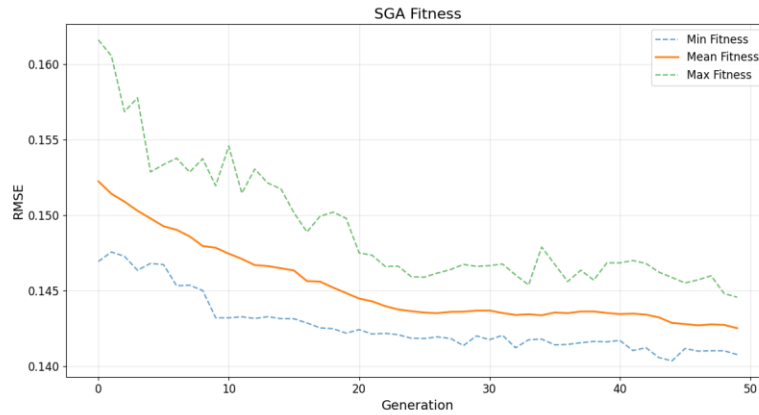


Figure 1 - Simple Genetic Algorithm fitness plot with all features

The two other survival selection methods used probabilistic and deterministic crowding, with Hamming distance being used as a distance measure to pair parents and children. Probabilistic crowding (PC) and deterministic crowding produced very similar results, so only PC was included in the plots. Figure 2 shows the results, with blue being SGA, orange being PC and green being PC with elitism. The three plots showcase the Mean Max and Min of the Fitness functions. From this figure we can see that all algorithms see improvement, but PC appears to converge quicker than SGA. Figure 3 showcases the Entropy a measurement of diversity of the population over the generations. Here we see that the entropy for SGA ends up being higher than the PC approaches. This was a surprising result, as crowding typically leads to higher diversity than generational. However, this could be a result of not running the algorithm for long enough, as both figures 2 and 3 show that PC with elitism converges significantly quicker than the other methods used. Due to the time-consuming nature of the problem, we could not explore this in greater detail.

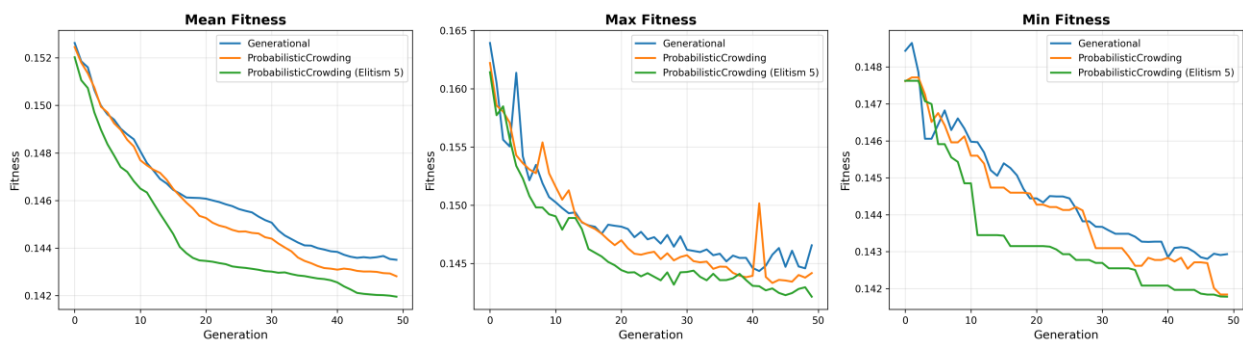


Figure 2: Max-min-mean plot for Generational, Probabilistic and Probabilistic with Elitism

As mentioned, elitism was implemented with PC and compared to the other methods. As seen in figure 2, the method implementing elitism seemed to converge quicker than the other approaches. Figure 3 shows that PC with elitism leads to a lower diversity, which is also an expected result.

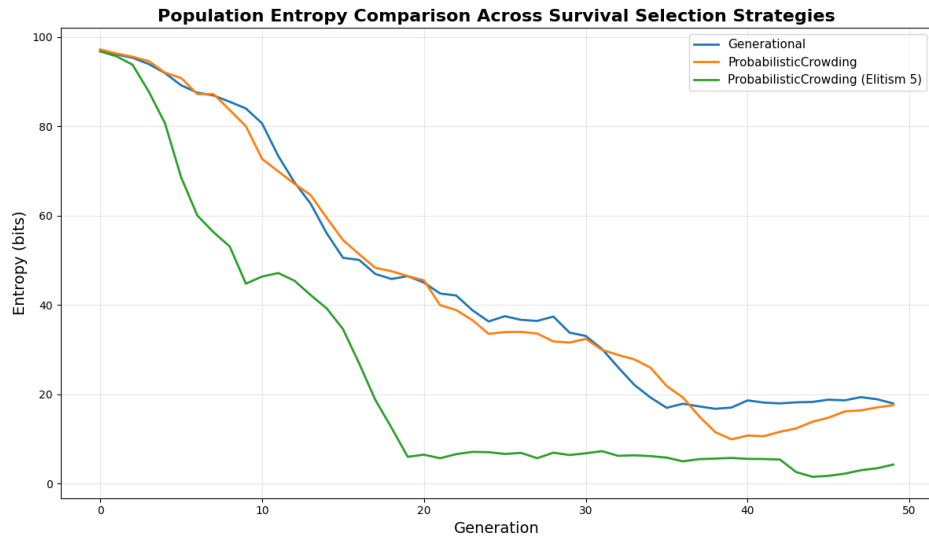


Figure 3: Entropy Plot

Conclusion

By analyzing the results could one derive that elitism although lowering diversity drastically more than the Generational and Probabilistic Crowding, resulting in better fitness. The results show that the difference in entropy between Generational and Probabilistic Crowding is not significant, this can be the result of a rather small population size. When comparing the mean-fitness is Probabilistic Crowding resulting in better fitness through the generations. In conclusion, the best approach combining Crowding with Elitism to ensure good enough diversity and fast convergence on a solution.

References

Díaz, X. F. C. S. (2026). 03-BioAI_2026_01_22, *IT3708 Bio-Inspired Artificial Intelligence*. Available at: [blackboard](#) (Accessed: 06/02/2026).

Mengshoel, O. J. (2026). 02-BioAI_2026_01_15_v1, *IT3708 Bio-Inspired Artificial Intelligence*. Available at: [blackboard](#) (Accessed: 06/02/2026).